

Lecture 16: Tree-Based Models

*Instructor: Jonathan Pipping-Gamón**Authors: JP, RB*

16.1 Expected Points in Football

Consider NFL play-by-play data, where each row is one play. Before the play starts, we observe a game state: field position, down, yards to go, time remaining, timeouts, and team strength. Let X collect those features.

The outcome Y is the next score from the offense's perspective. For example, $Y = 7$ if the offense scores the next touchdown, $Y = 3$ if it makes the next field goal, $Y = -3$ if the opponent makes the next field goal, and $Y = -7$ if the opponent scores the next touchdown. Safeties are treated the same way, with sign depending on which team scores. The expected-points target is the conditional mean

$$\text{EP}(x) = \mathbb{E}[Y \mid X = x].$$

So EP asks: among similar game states, how many points should the offense expect to score next, net of what it may allow?

This is a difficult regression problem because the structure is not close to linear:

- field position has nonlinear effects;
- down and distance interact strongly;
- thresholds matter near field-goal range, goal-to-go, and fourth down;
- time, timeouts, and team strength change incentives.

A linear regression can represent some of this behavior, but only after we choose transformations, indicators, and interaction terms. The harder part is specification. We would need to decide in advance which nonlinearities and which two-way, three-way, or higher-order interactions matter. For expected points, those interactions are exactly the patterns we would rather learn from the data.

Tree-based models take a different approach. They learn rules such as “yardline below this cutoff” or “third down with more than this many yards to go,” then average outcomes within the resulting groups. In that sense, trees learn threshold-based interactions automatically.

16.2 Regression Trees

A regression tree predicts by asking a sequence of yes/no questions. Each question splits the data into two groups. After enough splits, a new play falls into a terminal leaf, and the prediction is the average outcome among training plays in that leaf.

Definition 16.1 (Regression Tree). Suppose a tree partitions the predictor space into terminal leaves $R_1, \dots, R_M$. The fitted regression tree is

$$\hat{f}(x) = \sum_{m=1}^M \hat{c}_m \mathbb{1}\{x \in R_m\}, \quad \hat{c}_m = \frac{1}{|R_m|} \sum_{i: x_i \in R_m} y_i.$$

A tree can therefore be read as a flowchart. Internal nodes ask questions about the game state, and terminal leaves return predictions.

Regression Tree for Expected Points

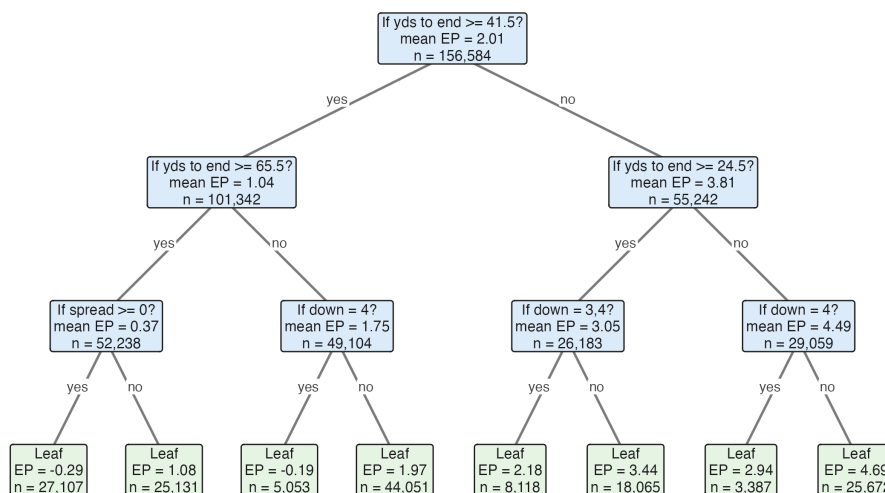


Figure 16.1: A shallow regression tree fit to play-by-play data.

For expected points, one terminal leaf might collect plays with the offense inside the opponent's 20, on first or second down, with fewer than 10 yards to go. The fitted EP value for that leaf is the average next-score value among training plays in that group.

16.2.1 How Trees Choose Splits

Trees are usually built by **recursive binary splitting**. At a node containing observations A , the algorithm searches over candidate splits. A split chooses a variable j and a cutoff s :

$$L(j, s) = \{i \in A : x_{ij} \leq s\}, \quad R(j, s) = \{i \in A : x_{ij} > s\}.$$

For a numerical predictor, the cutoffs are usually placed between observed values. For a categorical predictor, the candidates are binary groupings of levels.

For squared-error regression, the best split is the one with the smallest within-child-node residual sum of squares:

$$RSS(j, s) = \sum_{i \in L(j, s)} (y_i - \bar{y}_L)^2 + \sum_{i \in R(j, s)} (y_i - \bar{y}_R)^2,$$

where $\bar{y}_L$ and $\bar{y}_R$ are the average outcomes in the left and right child nodes.

Equivalently, the tree chooses the split that produces the largest reduction in within-node variance. A useful split separates game states with different average EP values, making the child nodes more homogeneous than the parent node.

The search is greedy. Once the tree chooses a split, it does not go back and revise that split after seeing what happens lower in the tree. The same procedure is then repeated separately inside each child node.

16.3 What a Single Tree Buys Us

Figure 16.2 shows a regression tree fit to the expected-points data. The points are binned historical averages. The step functions are fitted EP values at 10 yards to go, first half, 15:00 left, 3 timeouts each, and possession team spread 0.

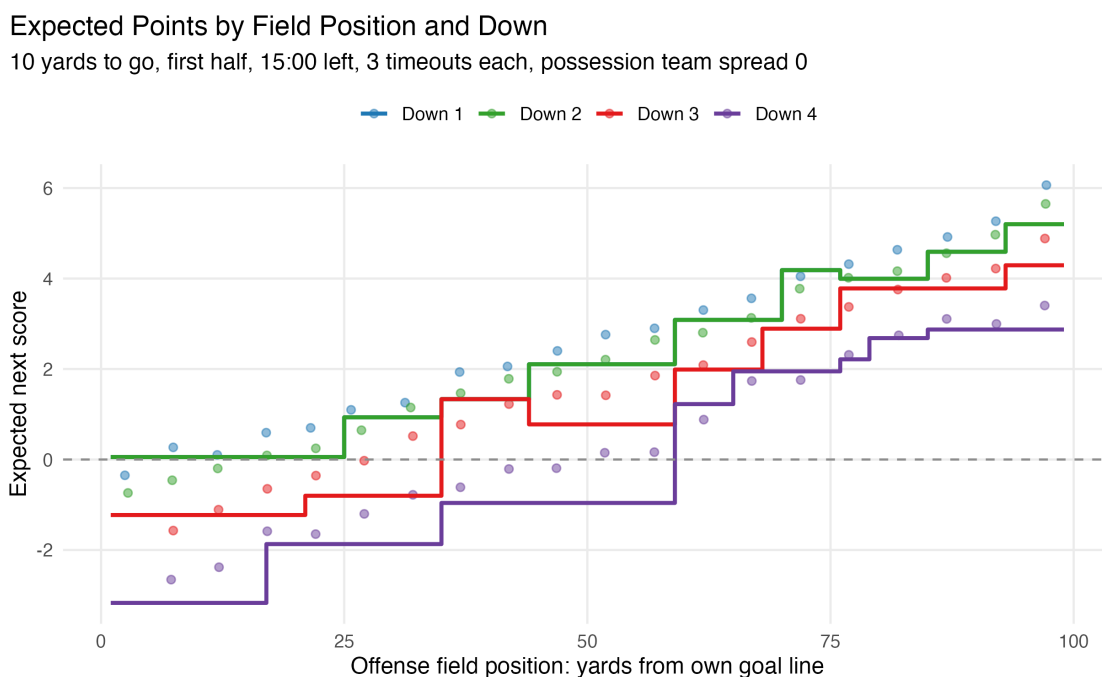


Figure 16.2: Observed binned averages and fitted regression tree expected points by field position and down.

The advantage is visible: the tree adapts to thresholds and interactions without requiring us to write them down in advance. Field position does not have to enter as one smooth slope, and down can change the shape of the fitted EP curve.

The cost is also visible. A tree is piecewise constant, so its predictions are blocky. Small changes in the training data can move split points and change the fitted tree. This is the main weakness of a single tree: it is flexible, interpretable, and often high variance.

Overfitting is the extreme version of this problem. A deeper tree creates smaller leaves. Smaller leaves reduce training error, but they can also chase rare plays: a missed field goal, a defensive touchdown, or an unusual end-of-half sequence. Lower training error is not the same as better prediction on future games.

16.4 Controlling Tree Complexity

Tree complexity controls how finely the model partitions the game-state space. Common controls include:

- **Maximum depth:** how many split levels the tree can have.
- **Minimum split size:** how many observations must be in a node before it can split.
- **Minimum leaf size:** how many observations must remain in each terminal leaf.
- **Cost-complexity pruning:** how strongly we penalize trees with many leaves.

Depth plays a role similar to λ in ridge regression: it controls the bias-variance tradeoff. Shallow trees are stable but may miss important structure. Deep trees are flexible but can overfit. The right level of complexity should be chosen with validation data.

Figure 16.3 shows the pattern to watch for when comparing possible tree depths. Training error keeps improving as depth increases. Validation error improves at first, then flattens out or worsens. The validation curve, not the training curve, should determine the complexity choice.

Training and Validation RMSE by Tree Depth

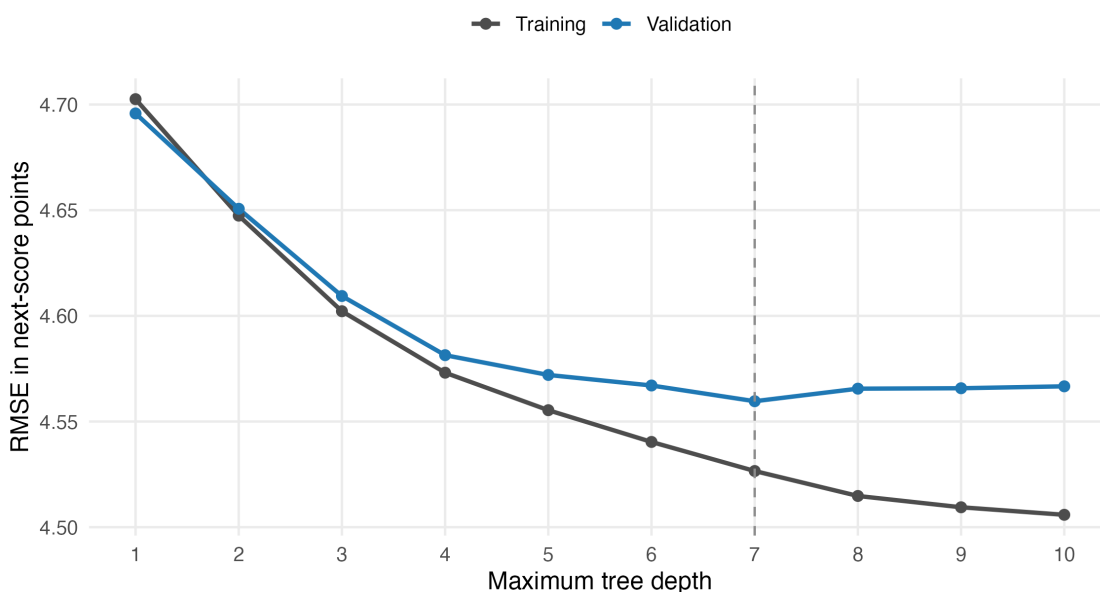


Figure 16.3: Training and validation RMSE by maximum tree depth.

16.5 Bagging and Random Forests

A single tree is unstable because even small data changes can dramatically alter its split structure. Bagging and random forests keep the flexibility of trees but reduce variance by averaging many trees.

16.5.1 Bagging: Bootstrap Aggregating

Bagging means **bootstrap aggregating**. For $b = 1, \dots, B$:

1. Draw a bootstrap sample from the training data.
2. Fit a tree $\hat{f}^{(b)}$ to that bootstrap sample.
3. Predict by averaging the fitted trees:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x).$$

Averaging helps because errors cancel. If the individual trees were independent and each had variance σ^2 , their average would have variance σ^2/B . In practice the trees are not independent, but averaging still reduces variance as long as their errors are not perfectly correlated.^a

16.5.2 Random Forests

Bagged trees can still be too similar. If every tree is allowed to use every predictor at every split, dominant predictors such as field position and down may appear near the top of almost every tree. Then the trees are averaged, but their errors remain correlated.

A **random forest** adds one more source of randomness. At each split, the algorithm considers only m randomly selected predictors rather than all p predictors. This makes the trees less redundant. For regression, the final prediction is still the average prediction across trees.

Random Forest Prediction

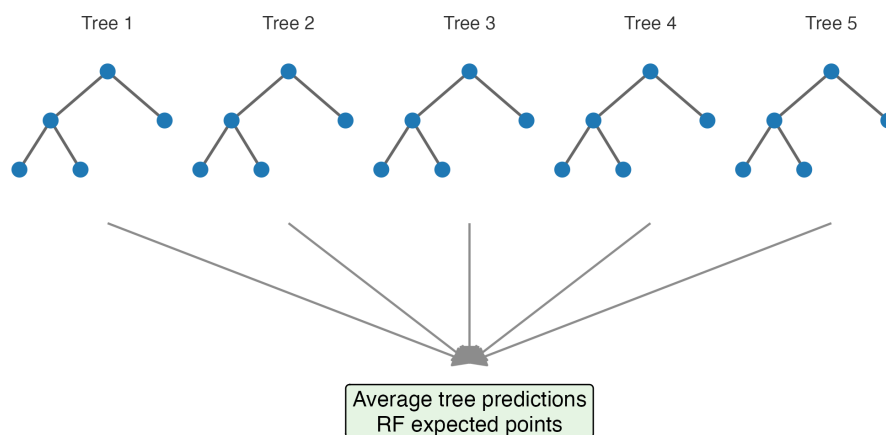


Figure 16.4: Random forest prediction from an average of trees.

^aFor play-by-play data, plays within the same game are dependent. If the bootstrap is used for uncertainty rather than prediction, resampling games is often better than resampling individual plays.

Figure 16.5 shows the same expected-points curve idea for a fitted random forest. Compared with a single tree, the predictions are less blocky because they average many different partitions of the game-state space.

Random Forest Expected Points by Field Position and Down

10 yards to go, first half, 15:00 left, 3 timeouts each, possession team spread 0

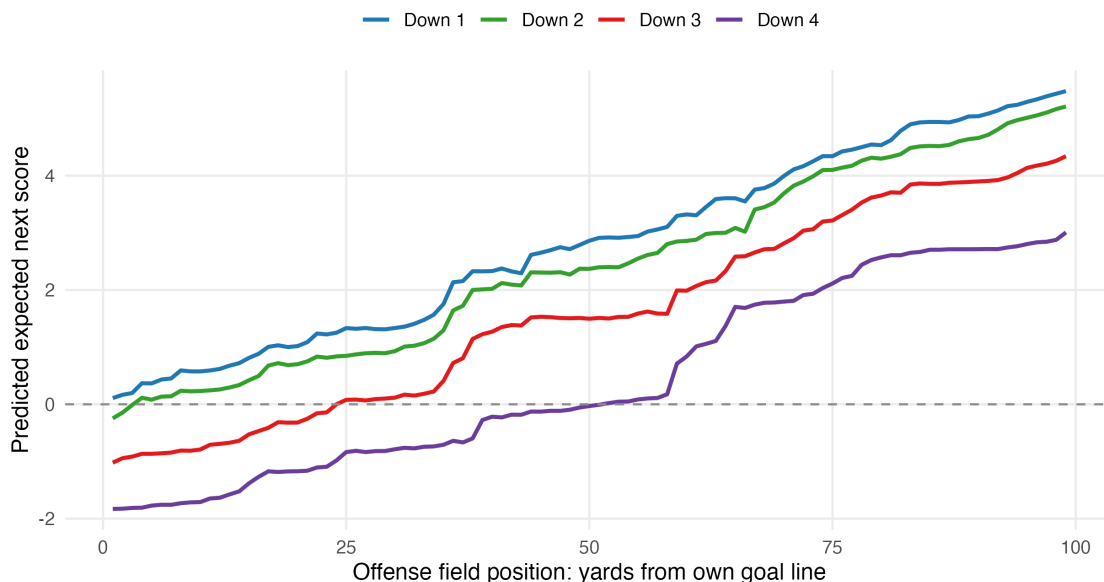


Figure 16.5: Random forest expected points predictions by field position and down.

The main tuning parameters are the number of trees B , the number of candidate predictors m considered at each split, minimum node or leaf size, and maximum allowed depth. For the random forest example, validation chooses m and minimum node size.

Random forests often predict better than a single tree, but they are less interpretable. There is no single sequence of splits to inspect. Instead, we summarize the fitted model with held-out prediction error, variable importance, and partial dependence plots.

16.6 Boosted Trees

Random forests fit many trees independently and average them. **Boosting** fits trees sequentially. Each new tree is trained to improve the current model.

For squared-error regression, start with the training mean:

$$\hat{f}_0(x) = \bar{y}.$$

At step k , compute the residuals left by the current model:

$$r_i^{(k)} = y_i - \hat{f}_{k-1}(x_i).$$

Fit a small tree $\hat{h}_k(x)$ to those residuals, then update

$$\hat{f}_k(x) = \hat{f}_{k-1}(x) + \eta \hat{h}_k(x),$$

where $0 < \eta \leq 1$ is the learning rate.

After K boosting rounds,

$$\hat{f}_K(x) = \hat{f}_0(x) + \eta \sum_{k=1}^K \hat{h}_k(x).$$

The distinction is worth keeping clear:

Random forests reduce variance by averaging many independently fit trees. Boosting builds one additive model by fitting new trees to what the current model still gets wrong.

Modern implementations such as XGBoost add regularization, subsampling, and efficient split-search algorithms. The core idea, however, is the sequential correction step above.

Important tuning parameters include boosting rounds K , learning rate η , tree depth, child weight, and row or column subsampling. For these examples, we validate a small grid over the main settings, with early stopping choosing K within each grid point. Too many boosting rounds can overfit, especially with deep trees or a large learning rate.

Boosted Tree Expected Points by Field Position and Down

10 yards to go, first half, 15:00 left, 3 timeouts each, possession team spread 0

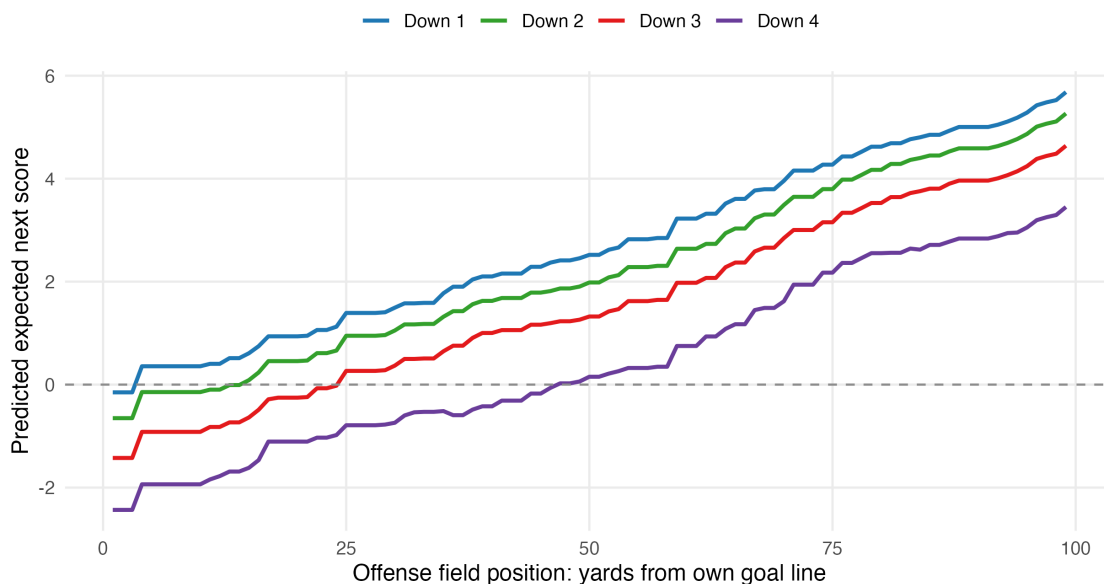


Figure 16.6: Boosted tree expected points predictions by field position and down.

16.7 Evaluation on Future Seasons

Model evaluation should reflect the prediction problem we actually care about: performance on future football games. Randomly holding out plays from the same season can be overly optimistic, because nearby plays, teams, coaches, and game environments are not independent. A more honest design is to split the data by season. Here we use the 2021–2025 seasons, skipping the COVID-disrupted 2020 season:

- train on 2021–2023 plays;
- validate on 2024 plays;
- test on 2025 plays.

The validation season chooses tree depth for the single tree, m and node size for the random forest, and grid settings plus early-stopped rounds for boosted trees. The test season is used only once, after those choices have been fixed, to estimate how well the final model predicts future games.

Figure 16.7 compares a mean-only baseline, a linear model,^b a single regression tree, a random forest, and boosted trees. In this split, boosted trees have the lowest test RMSE, but the linear model and random forest are close. That is an important lesson: flexible methods are not automatically better. Extra flexibility is useful only if it improves future-season prediction.

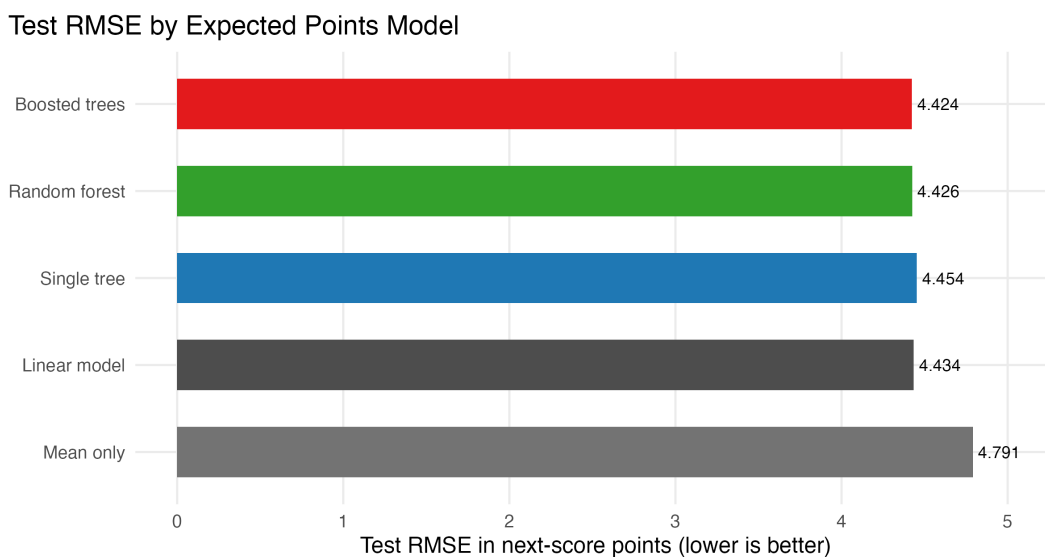


Figure 16.7: Test-season RMSE by expected-points model.

16.8 Optional Extension: Classification Trees

So far we have predicted EP directly as a conditional mean. Another option is to predict the full next-score distribution and compute EP from that distribution.

Let

$$C_i \in \mathcal{S} = \{-7, -3, -2, 0, 2, 3, 7\}$$

be the next-score class. A classification model estimates

$$p_s(x) = \mathbb{P}(C = s \mid X = x), \quad s \in \mathcal{S}.$$

Expected points can then be recovered as

$$\text{EP}(x) = \sum_{s \in \mathcal{S}} s \cdot p_s(x).$$

^bThe linear model uses `yardline_100` interacted with `down`, plus `ydstogo`, `half`, time remaining in the half, timeouts, and possession-team spread.

Definition 16.2 (Classification Tree). Suppose a tree partitions the predictor space into leaves $R_1, \dots, R_M$. For leaf m , the fitted class probability for score class s is

$$\hat{p}_{m,s} = \frac{1}{|R_m|} \sum_{i: x_i \in R_m} \mathbb{1}\{C_i = s\}.$$

For a new game state $x \in R_m$, the tree predicts the class distribution

$$\hat{p}_s(x) = \hat{p}_{m,s}, \quad s \in \mathcal{S}.$$

One common split criterion for classification trees is **Gini impurity**. If a node has class proportions $p_1, \dots, p_K$, its impurity is

$$\text{Gini} = 1 - \sum_{k=1}^K p_k^2.$$

Impurity is 0 when every observation in the node has the same class. For a candidate split of node A into children L and R , the score is

$$\frac{|L|}{|A|} \text{Gini}(L) + \frac{|R|}{|A|} \text{Gini}(R).$$

The tree picks the split with the lowest weighted child impurity, or equivalently the largest drop from the parent-node impurity.

Figure 16.8 shows the classification version of the expected-points problem. At each field position and down, the model predicts a distribution over possible next scores. EP is the weighted average of that distribution, but the full distribution can be more informative. Two states can have similar EP with very different touchdown, field-goal, turnover, and no-score probabilities.

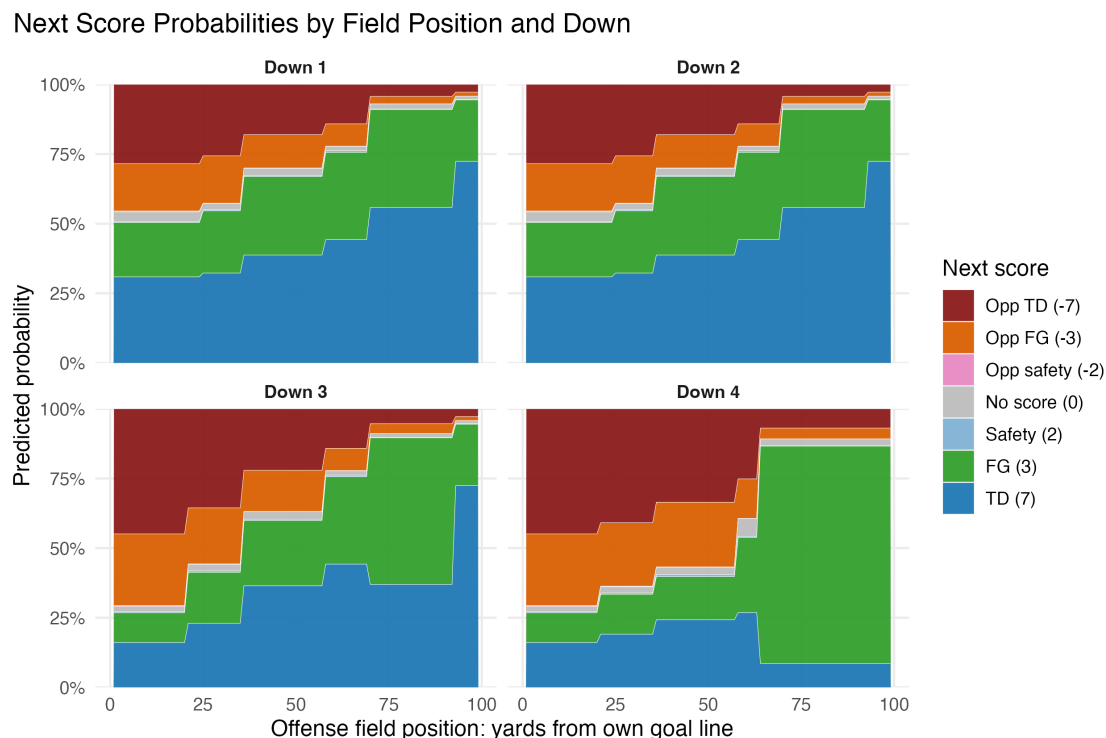


Figure 16.8: Predicted next score class probabilities by field position and down.

For classification models, the metric depends on the target. If we care about the most likely next score, use accuracy. If we care about the full distribution, use log loss and calibration. If we convert class probabilities back into expected points using $\sum_s s\hat{p}_s(x)$, evaluate the resulting EP predictions using RMSE.

16.9 Practical Takeaways

- Expected points is a conditional-mean regression target: $EP(x) = \mathbb{E}[Y \mid X = x]$.
- A regression tree estimates EP by partitioning the predictor space and averaging outcomes inside terminal leaves.
- Regression trees choose splits by reducing squared error, equivalently by reducing within-node variance.
- Single trees learn thresholds and interactions automatically, but they are high-variance predictors.
- Depth, minimum leaf size, and pruning control tree flexibility and should be chosen with validation data.
- Bagging reduces variance by averaging many bootstrap trees.
- Random forests improve bagging by decorrelating trees before averaging.
- Boosted trees build an additive model sequentially, fitting each new tree to the current model's errors.
- Evaluation should match the prediction task. For sports play-by-play models, time-based or game-based splits are usually more honest than random row splits.
- Tree models are flexible, but flexibility has to improve out-of-sample prediction to be worth using.

References

- [Breiman1996] Breiman, L., *Bagging Predictors*, Machine Learning, Vol. 24, pp. 123–140, 1996.
- [Breiman2001] Breiman, L., *Random Forests*, Machine Learning, Vol. 45, pp. 5–32, 2001.
- [Friedman2001] Friedman, J. H., *Greedy Function Approximation: A Gradient Boosting Machine*, The Annals of Statistics, Vol. 29, No. 5, pp. 1189–1232, 2001.
- [ChenGuestrin2016] Chen, T., & Guestrin, C., *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.